Security Review Report
NM-0799 - Yieldseeker

NETHERMIND SECURITY

(March 05, 2026)

Contents

1 Executive Summary **2**

2 Audited Files **3**

3 Summary of Issues **3**

4 System Overview **4**
4.1 Protocol Architecture . . . 4
4.1.1 High-Level Component Diagram . . . 4
4.2 Core Components . . . 5

5 Risk Rating Methodology **6**

6 Issues **7**
6.1 [Low] Malicious Agent Operator can drain wallet funds via arbitrary `swapCallData` in the `AWKZeroXAdapter` . . . 7
6.2 [Low] Untracked Vault Positions Cause Destructive Withdrawal Behavior and Fee Accounting Corruption . . . 8
6.3 [Info] FeeTracker Migration Orphans Historical Fee Debt and Position Accounting, Leading to Protocol Fee Loss . . . 9
6.4 [Info] No wallet-enforced time validity for 0x swaps . . . 9

7 Documentation Evaluation **10**

8 Test Suite Evaluation **11**
8.1 Tests Output . . . 11
8.2 Automated Tools . . . 17
8.2.1 AuditAgent . . . 17

9 About Nethermind **18**

1 Executive Summary

This document presents the results of a security review conducted by Nethermind Security for **Yieldseeker** protocol, a DeFi yield aggregation platform. **YieldSeeker** enables users to deploy autonomous agent wallets that interact with multiple lending protocols including but not limited to Aave V3, Compound V2, Compound V3 through a modular adapter architecture, while tracking fees on realized yield through a centralized `FeeTracker` contract.

The audit focused on the core smart contract infrastructure, including the agent wallet implementation (ERC-4337 account abstraction), the adapter registry system, protocol-specific adapters, and the fee tracking mechanism.

The audit comprises 1149 lines of the Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 4 points of attention, where two are classified as `Low` and two are classified as `Informational`. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the system overview. Section 4 discusses the risk rating methodology. Section 5 details the issues. Section 6 discusses the documentation provided by the client for this audit. Section 7 presents the test suite evaluation and automated tools used. Section 8 concludes the document.

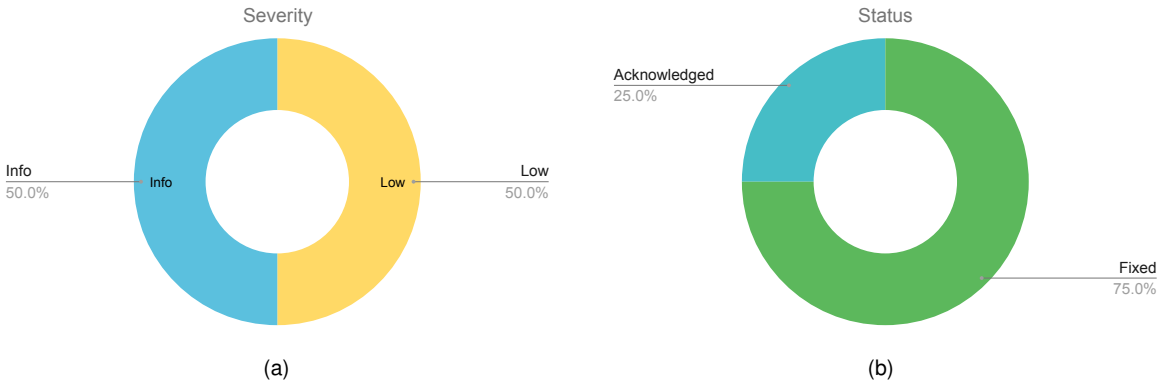


Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (0), **Low** (2), **Undetermined** (0), **Informational** (2), **Best Practices** (0). **Distribution of status: Fixed** (0), **Acknowledged** (0), **Mitigated** (0), **Unresolved** (4)

Summary of the Audit

Audit Type	Security Review
Initial Report	February 09, 2026
Final Report	March 05, 2026
Initial Commit	`95ce6c`
Final Commit	`0856fd`
Documentation Assessment	Medium
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/AgentWalletV1.sol	81	64	79.0%	27	172
2	src/FeeTracker.sol	114	92	80%	22	228
3	src/IAgentWalletFactory.sol	6	22	366%	2	30
4	src/AgentWalletFactory.sol	24	36	150%	6	66
5	src/AdapterRegistry.sol	5	23	460%	2	30
6	src/AdminTimelock.sol	5	25	500%	2	32
7	src/IAgentWallet.sol	8	23	287%	3	34
8	src/agentwalletkit/AWKAgentWalletFactory.sol	77	66	85%	15	157
9	src/agentwalletkit/AWKAdapter.sol	19	34	178%	7	60
10	src/agentwalletkit/IAWKAgentWalletFactory.sol	6	22	366%	3	31
11	src/agentwalletkit/AWKAdapterRegistry.sol	86	67	77%	17	168
12	src/agentwalletkit/IAWKAgentWallet.sol	2	19	950%	1	22
13	src/agentwalletkit/AWKErrors.sol	10	33	330%	11	54
14	src/agentwalletkit/AWKErrors.sol	232	176	75%	59	464
15	src/agentwalletkit/agentwalletkit/IAWKAdapter.sol	4	24	600%	1	29
16	src/agentwalletkit/adapters/AWKMerklAdapter.sol	26	35	134%	6	67
17	src/agentwalletkit/adapters/AWKBaseVaultAdapter.sol	47	72	153%	13	132
18	src/agentwalletkit/adapters/AWKCompoundV3Adapter.sol	38	38	100%	6	82
19	src/agentwalletkit/adapters/AWKZeroXAdapter.sol	52	42	80%	12	106
20	src/agentwalletkit/adapters/AWKCompoundV2Adapter.sol	36	38	105%	6	80
21	src/agentwalletkit/agentwalletkit/adapters/AWKERC4626Adapter.sol	28	35	125%	6	69
22	src/agentwalletkit/adapters/AWKAaveV3Adapter.sol	38	42	110%	7	87
23	src/adapters/MerklAdapter.sol	52	29	55%	2	79
24	src/agentwalletkit/AWKErrors.sol	30	35	116%	4	69
25	src/agentwalletkit/AWKErrors.sol	11	25	227%	3	39
26	src/adapters/CompoundV2Adapter.sol	39	41	105%	5	85
27	src/agentwalletkit/AWKErrors.sol	23	22	95%	7	52
28	src/adapters/ERC4626Adapter.sol	18	30	166%	3	51
29	src/adapters/CompoundV3Adapter.sol	32	35	109%	4	71
	Total	**1149**	**1245**	**108.4%**	**262**	**2646**

3 Summary of Issues

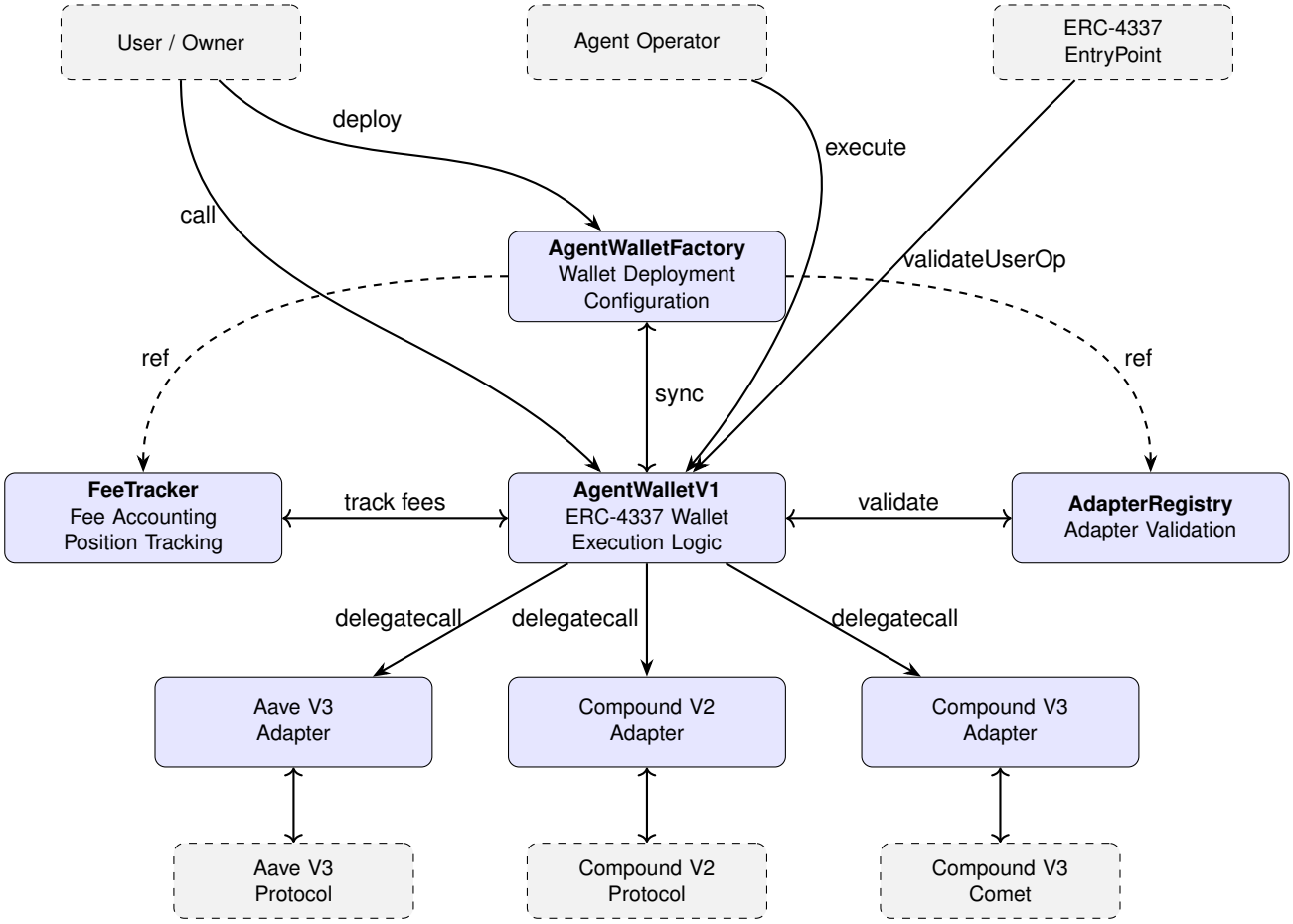
	Finding	Severity	Update
1	Malicious Agent Operator can drain wallet funds via arbitrary `swapCallData` in the `AWKZeroXAdapter`	Low	Fixed
2	Untracked Vault Positions Cause Destructive Withdrawal Behavior and Fee Accounting Corruption	Low	Fixed
3	FeeTracker Migration Orphans Historical Fee Debt and Position Accounting, Leading to Protocol Fee Loss	Info	Acknowledged
4	Info: No wallet-enforced time validity for 0x swaps	Info	Fixed

4 System Overview

4.1 Protocol Architecture

YieldSeeker is a modular DeFi yield aggregation protocol built on three foundational pillars: **agent wallets**, **protocol adapters**, and **fee tracking**. The architecture enables users to deploy ERC-4337 compliant smart contract wallets that autonomously interact with multiple lending protocols while the system tracks and charges performance fees on realized yield.

4.1.1 High-Level Component Diagram



The diagram illustrates three main interaction patterns:

1. Wallet Deployment & Configuration

- User → Factory: Deploys minimal proxy wallet
- Factory ↔ Wallet: Syncs registry & fee tracker references
- Result: User owns an upgradeable smart contract wallet

2. Protocol Execution Flow

- Owner/Operator/EntryPoint → Wallet: Execution request arrives
- Wallet → AdapterRegistry: Validates adapter approval for target
- Wallet $\xrightarrow{\text{delegatecall}}$ Adapter: Transfers execution context
- Adapter ↔ External Protocol Adapter → FeeTracker: Deposits/withdraws/swaps then records positions and fees.

3. Fee Enforcement Loop

- Adapters write: Position data during protocol interactions
- Wallet reads: Fee obligations before allowing withdrawals
- Result: Users cannot withdraw without reserving owed fees

4.2 Core Components

The YieldSeeker protocol is built on three foundational components that work together to enable autonomous yield aggregation with transparent fee tracking:

1. Agent Wallets: Smart contract wallets that serve as the user's interface to the protocol, combining account abstraction with fee enforcement:

- **ERC-4337 Account Abstraction:** Each wallet is a smart contract implementing the ERC-4337 standard, enabling gasless transactions through bundlers and flexible signature validation beyond traditional externally owned accounts
- **Factory-Based Deployment:** The AgentWalletFactory deploys minimal proxy wallets pointing to a shared implementation contract, enabling efficient deployment while maintaining upgradeability through the UUPS pattern
- **Multi-Role Authorization:** Wallets support three authorization tiers — wallet owners have full control including withdrawals and upgrades, agent operators can execute protocol interactions on behalf of users, and the ERC-4337 EntryPoint validates and executes signed user operations
- **Fee-Aware Withdrawals:** Unlike standard wallets, YieldSeeker wallets enforce fee obligations at the withdrawal layer by querying the FeeTracker and reserving owed fees before allowing asset transfers to users
- **User Sovereignty Features:** Wallet owners can block specific adapters or protocol targets even if they are globally approved, providing an emergency brake against malicious or compromised adapters
- **Configuration Synchronization:** Wallets can pull updated registry and fee tracker references from the factory through a sync mechanism, enabling protocol-wide upgrades without individual wallet migrations

2. Protocol Adapters: Modular integration layers that enable wallets to interact with external DeFi protocols in a controlled, auditable manner:

- **Delegatecall Execution Model:** Adapters are pure logic contracts executed via delegatecall from the wallet's context, meaning they operate with the wallet's storage and token balances but cannot be called directly
- **Registry-Based Validation:** Before any adapter executes, the wallet validates that the adapter is registered for the specific target contract (vault, pool, etc.) in the global AdapterRegistry, preventing unauthorized code execution
- **Protocol-Specific Implementations:** Each supported DeFi protocol (Aave V3, Compound V2, Compound V3, 0x) has a dedicated adapter that understands its unique interface and token model, abstracting complexity from the wallet layer
- **Hierarchical Design:** Adapters extend from base contracts that provide common functionality (vault deposits/withdrawals, token swaps), with protocol-specific implementations layering on top to handle rebasing tokens, exchange rates, and yield tracking
- **Fee Tracking Integration:** YieldSeeker adapters wrap generic adapter logic with calls to the FeeTracker, recording deposits to establish cost basis and withdrawals to calculate realized yield and corresponding fees
- **Security Isolation:** Each adapter enforces that it can only be called via delegatecall by checking whether the execution context address matches its own deployment address, preventing direct invocation that could bypass wallet authorization

3. Fee Tracking System: A centralized accounting layer that tracks per-wallet fee obligations and vault positions to enable accurate performance fee calculation on realized yield:

- **Virtual Share Position Tracking:** To handle rebasing tokens (Aave aTokens, Compound V3 Comet balances) where the token balance grows over time, the tracker records a fixed virtual share amount at deposit time representing the user's cost basis
- **Proportional Yield Calculation:** On withdrawal, adapters query the virtual share position and convert the requested shares into actual protocol tokens using a proportional formula, enabling the system to distinguish between principal and yield even for rebasing tokens
- **Multi-Token Fee Obligations:** When yield is earned in non-base-asset tokens (such as protocol rewards), the tracker records fee obligations denominated in those tokens, then converts them to base asset fees when the tokens are swapped
- **Profit-Based Fee Model:** Fees are only charged on the profit portion of withdrawals by comparing the assets received against the proportional cost basis, ensuring users are not charged fees on their original principal
- **Centralized Reference Point:** The factory maintains a single FeeTracker address that all wallets sync to, creating a protocol-wide source of truth for fee configuration (rate, collector address) and per-wallet accounting state

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] Malicious Agent Operator can drain wallet funds via arbitrary swapCallData in the AWKZeroXAdapter

File(s): AWKZeroXAdapter.sol

Description: The AWKZeroXAdapter contract enables token swaps through the 0x protocol. When a swap is executed, the _executeAdapterCall(...) function in AWKAgentWalletV1 validates that the target address is registered in the adapter registry:

```
function _executeAdapterCall(...) private returns (bytes memory result) {
    // ...
    address registeredAdapter = adapterRegistry().getTargetAdapter(target);
    if (registeredAdapter == address(0) || registeredAdapter != adapter) {
        revert AWKErrors.AdapterNotRegistered(adapter);
    }
    // ...
}
```

While the target parameter is validated, the swapCallData passed to the adapter is fully controlled by the Agent Operator. In the function _swapInternal(...), the adapter approves the AllowanceHolder contract for the sellAmount of tokens and then executes a call to the registered target with operator-supplied calldata:

```
function _swapInternal(...) internal virtual returns (uint256 buyAmount, uint256 soldAmount) {
    if (sellAmount == 0 || minBuyAmount == 0) revert AWKErrors.ZeroAmount();
    // ...
    // @audit Approves AllowanceHolder for sellAmount.
    IERC20(sellToken).forceApprove(ALLOWANCE_TARGET, sellAmount);
    // ...
    // @audit The swapCallData is fully controlled by the operator.
    (bool success, bytes memory reason) = target.call{value: ethToSend}(swapCallData);
    // ...
    if (buyAmount < minBuyAmount) revert InsufficientOutput(buyAmount, minBuyAmount);
}
```

The 0x Exchange Proxy (the registered target) supports routing swaps through the AllowanceHolder contract as part of its normal operation. A malicious operator can craft swapCallData that:

1. Routes through AllowanceHolder.exec(...) with an attacker-controlled operator and target ;
2. The attacker's contract is called by AllowanceHolder and invokes AllowanceHolder.transferFrom(...) to transfer the wallet's tokens to an arbitrary recipient ;

Since the wallet already approved AllowanceHolder for sellAmount, the transferFrom(...) call succeeds. To bypass the output validation, the attacker sets minBuyAmount to 1 wei and ensures the swap returns at least 1 wei of buyToken. The result is that the wallet loses nearly all of sellAmount while receiving only dust in return.

Recommendation(s): Consider enforcing a minimum output amount on-chain using a trusted price oracle (e.g., Chainlink) with an owner-configured slippage tolerance, rather than relying on the operator-supplied minBuyAmount. This would ensure that no swap can settle significantly below fair market value, regardless of the swapCallData or minBuyAmount provided by a potentially compromised operator.

Status: Fixed

Update from the client: Fixed in PR 10.

6.2 [Low] Untracked Vault Positions Cause Destructive Withdrawal Behavior and Fee Accounting Corruption

File(s): `AaveV3Adapter.sol`, `CompoundV2Adapter.sol`, `CompoundV3Adapter.sol`, `FeeTracker.sol`

Description: The AaveV3, CompoundV2, and CompoundV3 adapters use a virtual share system to track fee-relevant positions. Each deposit records a 1:1 mapping of assets-to-virtual-shares in the `FeeTracker`. On withdrawal, the adapter queries `totalVirtualShares` from the tracker and uses it to convert the requested virtual shares into the actual protocol token amount to redeem.

When a withdrawal is requested for a vault where `totalVirtualShares == 0` — meaning the `FeeTracker` has no record of deposits — the proportional conversion is skipped entirely, and each adapter falls back to different but uniformly incorrect default behavior. This state arises when the wallet holds vault tokens that were received outside of a tracked deposit, such as via a direct ERC20 transfer of aTokens/cTokens to the wallet address, or as a downstream consequence of a FeeTracker migration.

Per-Adapter Breakdown

CompoundV2Adapter — Full position liquidation

```
// CompoundV2Adapter.sol:73-76
uint256 cTokensToRedeem = cTokenBalance;           // default: entire balance
if (totalVirtualShares > 0 && cTokenBalance > 0) {
    cTokensToRedeem = (cTokenBalance * virtualShares) / totalVirtualShares;
}
```

The default at line 73 sets `cTokensToRedeem` to the wallet's full `cTokenBalance`. When `totalVirtualShares == 0`, the `if` block is skipped. A request to withdraw 1 virtual share redeems every cToken the wallet holds.

Example: Wallet holds 1,000 cUSDC (worth $50,000). Operator requests withdrawal of 100 virtual shares. Because `totalVirtualShares == 0`, the adapter redeems all 1,000 cUSDC — a 10x over-withdrawal that withdraws the entire Compound V2 position.

AaveV3Adapter — Share-to-asset type confusion

```
// AaveV3Adapter.sol:60-63
uint256 aTokensToWithdraw = virtualShares;          // default: raw share units
if (totalVirtualShares > 0 && currentBalance > 0) {
    aTokensToWithdraw = (currentBalance * virtualShares) / totalVirtualShares;
}
```

The default at line 60 uses `virtualShares` directly as the withdrawal amount passed to `IAaveV3Pool.withdraw()`. Virtual shares are an internal accounting abstraction denominated in deposit-time units — they are not Aave pool asset amounts. The conversion that normally maps virtual shares to their current aToken value (accounting for accrued yield) is skipped.

Example: Wallet holds 1,050 aUSDC (1,000 deposited + 50 yield). If the operator issues a withdrawal for 500 virtual shares, the adapter passes `500` directly to Aave's `withdraw()` as an asset amount, instead of the correct proportional value of `(1050 * 500) / 1000 = 525`. The withdrawal underdraws by 25 USDC, and the yield portion is never realized.

CompoundV3Adapter — Identical type confusion

```
// CompoundV3Adapter.sol:60-63
uint256 amountToWithdraw = virtualShares;           // default: raw share units
if (totalVirtualShares > 0 && currentBalance > 0) {
    amountToWithdraw = (currentBalance * virtualShares) / totalVirtualShares;
}
```

Same pattern as AaveV3. The raw `virtualShares` value is passed to `ICompoundV3Comet.withdraw()` as an underlying asset amount.

Since any `sharesSpent > 0` exceeds `totalShares == 0`, this always enters the over-withdrawal branch. Because `totalShares == 0`, the fee-on-profit calculation is skipped entirely (the inner `if` guard fails), and both `agentVaultCostBasis` and `agentVaultShares` are reset to zero. If any future tracked deposits occur for this vault, they start from a clean but incorrect baseline — any yield earned on the untracked position is never subject to fees.

Recommendation(s): Consider checking if `totalVirtualShares == 0` and the wallet has non-zero vault token balance before proceeding with withdrawals.

Status: Fixed

Update from the client: Fixed in PR 8.

6.3 [Info] FeeTracker Migration Orphans Historical Fee Debt and Position Accounting, Leading to Protocol Fee Loss

File(s): AgentWalletFactory.sol, AgentWalletV1.sol, FeeTracker.sol

Description: When the admin calls YieldSeekerAgentWalletFactory.setFeeTracker() to replace the active FeeTracker, no historical state is migrated from the old tracker to the new one. The FeeTracker contract maintains per-wallet accounting across six mappings:

- agentFeesCharged / agentFeesPaid — fee ledger per wallet
- agentVaultCostBasis / agentVaultShares — vault position tracking per wallet per vault
- agentYieldTokenFeesOwed — non-base-asset fee obligations per wallet per token

The setter at AgentWalletFactory.sol:54-64 only validates the new address is non-zero and has deployed code, then replaces the reference:

```
function setFeeTracker(
    FeeTracker newFeeTracker
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (address(newFeeTracker) == address(0))
        revert AWKErrors.ZeroAddress();
    if (address(newFeeTracker).code.length == 0)
        revert AWKErrors.NotAContract(address(newFeeTracker));
    FeeTracker oldFeeTracker = feeTracker;
    feeTracker = newFeeTracker;
    emit TrackerUpdated(address(oldFeeTracker), address(newFeeTracker));
}
```

Once wallets call syncFromFactory(), the internal _syncFromFactory() at AgentWalletV1.sol:116-128 blindly adopts the new tracker. From that point forward, all fee queries (getFeesOwed, getAgentVaultPosition) are read from the new tracker which has empty state for every wallet.

This leads to fee revenue loss and virtual share position tracking is lost.

Recommendation(s): Consider implementing a migration mechanism that transfers critical accounting state from the old tracker to the new tracker during the setFeeTracker call.

Status: Acknowledged.

Update from the client: The team plans to handle state migration within the new FeeTracker contract itself, which will restore historical accounting data from the old tracker as part of the upgrade process. The team will document this in the upgrade procedure to ensure it is not overlooked.

6.4 [Info] No wallet-enforced time validity for 0x swaps

File(s): AgentWalletFactory.sol, AgentWalletV1.sol/, FeeTracker.sol

Description: The 0x swap adapter enforces output amount slippage via minBuyAmount , but there is no explicit time validity check enforced by the adapter itself, and the wallet's ERC-4337 signature validation does not set any validUntil / validAfter bounds. Wallet signature validation returns 0 for owner/operator signatures, which corresponds to no time range restrictions

The 0x swap flow itself does not check block timestamp against any deadline parameter:

```
function _swapInternal(
    address target,
    address sellToken,
    address buyToken,
    uint256 sellAmount,
    uint256 minBuyAmount,
    bytes memory swapCallData,
uint256 value
) internal virtual returns (uint256 buyAmount, uint256 soldAmount) {
...
    (bool success, bytes memory reason) = target.call{value: ethToSend}(swapCallData);
    ...
    if (buyAmount < minBuyAmount) revert InsufficientOutput(buyAmount, minBuyAmount);
}
```

As a result, a signed ERC-4337 UserOperation that performs a swap can potentially be executed significantly later than the quote time if it remains un-included, and the contract itself does not enforce that the operation must execute before a specific deadline.

Recommendation(s): Consider enforcing time validity for swaps inZeroX

Status: Fixed

Update from the client: Fixed in PR 10.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Yieldseeker's documentation

The documentation for the contracts was provided through the code's Natspec and README files. Moreover, the protocol's development team addressed all questions and concerns raised by the Nethermind Security team during the kick-off call and through asynchronous communication, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```
forge test
[] Compiling...
No files changed, compilation skipped

Ran 6 tests for test/unit/AdminTimelock.t.sol:YieldSeekerAdminTimelockTest
[PASS] test_Cancel_Succeeds() (gas: 46145)
[PASS] test_Constructor_RoleAssignment() (gas: 23902)
[PASS] test_Execute_EnforcesDelay() (gas: 92081)
[PASS] test_Execute_OnlyExecutor() (gas: 52408)
[PASS] test_ScheduleAndExecute_Succeeds() (gas: 90464)
[PASS] test_Schedule_OnlyProposer() (gas: 13507)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 709.46µs (197.17µs CPU time)

Ran 12 tests for test/unit/adapters/AaveV3Adapter.t.sol:AaveV3AdapterTest
[PASS] test_Execute_DepositPercentage_UsesBalance() (gas: 169038)
[PASS] test_Execute_DepositZeroAmount_Reverts() (gas: 23855)
[PASS] test_Execute_Deposit_InvalidAsset_Reverts() (gas: 752754)
[PASS] test_Execute_Deposit_Succeeds() (gas: 169218)
[PASS] test_Execute_WithdrawZeroShares_Reverts() (gas: 19962)
[PASS] test_Execute_Withdraw_Succeeds() (gas: 197013)
[PASS] test_FeeAccrual_PartialWithdraw_NoYield() (gas: 195283)
[PASS] test_FullWithdraw_ClearsCostBasis() (gas: 165300)
[PASS] test_PartialWithdraw_ProportionalCostBasis() (gas: 191929)
[PASS] test_SequentialDeposits_AccumulateCostBasis() (gas: 190059)
[PASS] test_VirtualShares_PartialWithdraw_WithYield() (gas: 232278)
[PASS] test_VirtualShares_YieldAccrual_FeeCharged() (gas: 199294)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 1.47ms (1.02ms CPU time)

Ran 6 tests for test/security/EdgeCaseSecurity.t.sol:EdgeCaseSecurityTest
[PASS] test_BatchAtomicityRevertsAndRollsBack() (gas: 608149)
[PASS] test_BatchSuccessAccumulatesShares() (gas: 596192)
[PASS] test_DepositZeroAmountReverts() (gas: 436756)
[PASS] test_FeeRateAboveMaxReverts() (gas: 15870)
[PASS] test_WithdrawWithoutSharesReverts() (gas: 397931)
[PASS] test_WithdrawZeroSharesReverts() (gas: 436891)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 1.45ms (592.37µs CPU time)

Ran 4 tests for test/security/ReentrancySecurity.t.sol:ReentrancySecurityTest
[PASS] test_NonExecutorCannotTriggerReenterAdapter() (gas: 417019)
[PASS] test_ReenterDuringBatch_RevertsUnauthorized() (gas: 608110)
[PASS] test_ReenterExecuteViaAdapter_RevertsUnauthorized() (gas: 429296)
[PASS] test_ReenterWithdrawBaseAsset_RevertsUnauthorized() (gas: 425319)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.01ms (647.54µs CPU time)

Ran 13 tests for test/unit/adapters/CompoundV2Adapter.t.sol:CompoundV2AdapterTest
[PASS] test_ExchangeRate_AffectsShareCalculation() (gas: 197382)
[PASS] test_Execute_DepositPercentage_UsesBalance() (gas: 166817)
[PASS] test_Execute_DepositZeroAmount_Reverts() (gas: 23754)
[PASS] test_Execute_Deposit_InvalidAsset_Reverts() (gas: 690180)
[PASS] test_Execute_Deposit_Succeeds() (gas: 167164)
[PASS] test_Execute_WithdrawZeroShares_Reverts() (gas: 19990)
[PASS] test_Execute_Withdraw_Succeeds() (gas: 194529)
[PASS] test_FeeAccrual_PartialWithdraw_NoYield() (gas: 192772)
[PASS] test_FullWithdraw_ClearsCostBasis() (gas: 163296)
[PASS] test_PartialWithdraw_ProportionalCostBasis() (gas: 189424)
[PASS] test_SequentialDeposits_AccumulateCostBasis() (gas: 186282)
[PASS] test_VirtualShares_PartialWithdraw_WithYield() (gas: 232309)
[PASS] test_VirtualShares_YieldAccrual_FeeCharged() (gas: 198488)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 1.29ms (959.34µs CPU time)

Ran 6 tests for test/security/RegistrySecurity.t.sol:RegistrySecurityTest
[PASS] test_PauseReturnsNoAdapter() (gas: 451838)
[PASS] test_RemoveTargetAfterUnregisterPreventsReactivation() (gas: 450322)
[PASS] test_RemoveTargetBlocksEvenIfAdapterRegistered() (gas: 435121)
[PASS] test_ReregisterAdapterRestoresExecution() (gas: 583497)
[PASS] test_UnpauseRestoresAdapterLookup() (gas: 582601)
[PASS] test_UnregisteredAdapterBlocksExecution() (gas: 435228)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.40ms (1.06ms CPU time)
```

```
Ran 8 tests for test/integration/AdapterIntegration.t.sol:AdapterIntegrationTest
[PASS] test_AdapterBlockingPreventsAllOperations() (gas: 589559)
[PASS] test_AdapterRegistrationAfterWalletCreation() (gas: 2133646)
[PASS] test_ConsolidateFromMultipleVaults() (gas: 836565)
[PASS] test_DistributionAcrossMultipleVaults() (gas: 849281)
[PASS] test_ERC4626_DepositThenAppreciateThenWithdraw_ChargesFees() (gas: 519352)
[PASS] test_MultipleUsersIndependentPositions() (gas: 1008779)
[PASS] test_RegistryAdapterRemoval_AffectsActiveWallets() (gas: 580365)
[PASS] test_SequentialDepositsAndWithdrawals() (gas: 605745)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 2.81ms (1.68ms CPU time)

Ran 8 tests for test/security/AccessControlSecurity.t.sol:AccessControlSecurityTest
[PASS] test_AdminCanUpdateFeeConfig() (gas: 26503)
[PASS] test_NonAdminCannotSetAdapterRegistry() (gas: 780295)
[PASS] test_NonOperatorCannotCreateWallet() (gas: 18161)
[PASS] test_NonOwnerCannotBlockAdapter() (gas: 571694)
[PASS] test_OnlyOwnerCanExecuteViaAdapter() (gas: 575734)
[PASS] test_OperatorCreatesWalletSetsOwner() (gas: 360610)
[PASS] test_OwnerBlockAdapterPreventsExecution() (gas: 441718)
[PASS] test_RegistryPauseBlocksExecution() (gas: 452002)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 3.03ms (1.59ms CPU time)

Ran 12 tests for test/unit/adapters/CompoundV3Adapter.t.sol:CompoundV3AdapterTest
[PASS] test_Execute_DepositPercentage_UsesBalance() (gas: 162667)
[PASS] test_Execute_DepositZeroAmount_Reverts() (gas: 23737)
[PASS] test_Execute_Deposit_InvalidAsset_Reverts() (gas: 408010)
[PASS] test_Execute_Deposit_Succeeds() (gas: 162894)
[PASS] test_Execute_WithdrawZeroShares_Reverts() (gas: 19963)
[PASS] test_Execute_Withdraw_Succeeds() (gas: 188500)
[PASS] test_FeeAccrual_PartialWithdraw_NoYield() (gas: 186749)
[PASS] test_FullWithdraw_ClearsCostBasis() (gas: 158477)
[PASS] test_PartialWithdraw_ProportionalCostBasis() (gas: 183401)
[PASS] test_SequentialDeposits_AccumulateCostBasis() (gas: 180050)
[PASS] test_VirtualShares_PartialWithdraw_WithYield() (gas: 223654)
[PASS] test_VirtualShares_YieldAccrual_FeeCharged() (gas: 192376)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 1.09ms (817.67µs CPU time)

Ran 9 tests for test/unit/adapters/ERC4626Adapter.t.sol:ERC4626AdapterTest
[PASS] test_Deposit_PrecisionWithExistingShares() (gas: 191287)
[PASS] test_Execute_DepositPercentage_UsesBalance() (gas: 165688)
[PASS] test_Execute_DepositZeroAmount_Reverts() (gas: 23727)
[PASS] test_Execute_Deposit_InvalidAsset_Reverts() (gas: 1026380)
[PASS] test_Execute_Deposit_Succeeds() (gas: 165815)
[PASS] test_Execute_WithdrawZeroShares_Reverts() (gas: 23906)
[PASS] test_Execute_Withdraw_Succeeds() (gas: 189205)
[PASS] test_FeeAccrual_OnProfitableWithdraw() (gas: 187993)
[PASS] test_Withdraw_PrecisionWithExistingShares() (gas: 217502)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 806.63µs (516.17µs CPU time)

Ran 5 tests for test/unit/adapters/ZeroXAdapter.t.sol:ZeroXAdapterTest
[PASS] test_Execute_SwapTokens_Succeeds() (gas: 138390)
[PASS] test_Execute_Swap_NativeUsesSellAmount() (gas: 121876)
[PASS] test_Execute_Swap_RevertBubble() (gas: 87916)
[PASS] test_Execute_Swap_RevertsOnLowOutput() (gas: 150772)
[PASS] test_Execute_Swap_RevertsOnZeroAmounts() (gas: 26071)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 703.13µs (392.50µs CPU time)

Ran 4 tests for test/unit/adapters/MerklAdapter.t.sol:MerklAdapterTest
[PASS] test_Claim_BaseAssetRecordsFee() (gas: 100945)
[PASS] test_Claim_DuplicateTokens_DeduplicatesFeeTracking() (gas: 111569)
[PASS] test_Claim_MixedTokens_TracksSeparately() (gas: 166779)
[PASS] test_Claim_RevertPropagates() (gas: 61516)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 509.54µs (280.63µs CPU time)

Ran 36 tests for test/unit/FeeTracker.t.sol:FeeTrackerTest
[PASS] test_AccessControl_AdminOnly_SetFeeConfig() (gas: 16527)
[PASS] test_AccessControl_VaultDeposit_OnlyAdmin() (gas: 18649)
[PASS] test_AccessControl_VaultWithdraw_OnlyAdmin() (gas: 69823)
[PASS] test_AgentFeesTracking_MultipleAgentsIndependent() (gas: 72907)
[PASS] test_CalculateFeeAmount_MaxFeeRate() (gas: 25812)
[PASS] test_CalculateFeeAmount_RoundingEdgeCases() (gas: 8525)
[PASS] test_CalculateFeeAmount_StandardRate() (gas: 7673)
[PASS] test_CalculateFeeAmount_ZeroFeeRate() (gas: 20786)
[PASS] test_CalculateYield_MaxPrecision() (gas: 7854)
```

```
[PASS] test_FeeConfig_UpdateAffectsCalculations() (gas: 59753)
[PASS] test_GetFeeConfig_Correct() (gas: 12484)
[PASS] test_GetFeesOwed_AfterRecording() (gas: 43180)
[PASS] test_GetFeesOwed_NewAgent() (gas: 10623)
[PASS] test_RecordYield_Accumulation() (gas: 47936)
[PASS] test_RecordYield_MultipleAgents() (gas: 73338)
[PASS] test_RecordYield_OnlyAdmin() (gas: 16172)
[PASS] test_RecordYield_ValidAmount() (gas: 45195)
[PASS] test_RecordYield_ZeroAmount() (gas: 18872)
[PASS] test_SetFeeConfig_MaxFeeRate() (gas: 16125)
[PASS] test_SetFeeConfig_OnlyAdmin() (gas: 16008)
[PASS] test_SetFeeConfig_Success() (gas: 30443)
[PASS] test_SetFeeConfig_ZeroCollector() (gas: 14177)
[PASS] test_StateManagement_AccumulatedDeposits() (gas: 66732)
[PASS] test_StateManagement_MultipleAgentsIndependentVaults() (gas: 114589)
[PASS] test_StateManagement_MultipleVaultsPerAgent() (gas: 114152)
[PASS] test_TokenSwap_ZeroSwappedAmount_DoesNotRevert() (gas: 50372)
[PASS] test_VaultPositionMath_BasicDeposit() (gas: 63406)
[PASS] test_VaultPositionMath_MaxPrecision() (gas: 70272)
[PASS] test_VaultPositionMath_PartialWithdraw() (gas: 93688)
[PASS] test_VaultPositionMath_WithdrawLoss() (gas: 52163)
[PASS] test_VaultPositionMath_WithdrawProfit() (gas: 71435)
[PASS] test_VaultPositionMath_ZeroInitialDeposit() (gas: 27199)
[PASS] test_YieldTokenFees_PartialSwap() (gas: 71666)
[PASS] test_YieldTokenFees_RecordEarnings() (gas: 43574)
[PASS] test_YieldTokenFees_SwapMoreThanFeeOwed() (gas: 53350)
[PASS] test_YieldTokenFees_SwapToBaseAsset() (gas: 54151)
Suite result: ok. 36 passed; 0 failed; 0 skipped; finished in 2.93ms (2.45ms CPU time)

Ran 8 tests for test/security/FeeTrackerSecurity.t.sol:FeeTrackerSecurityTest
[PASS] test_FeePaymentReducesFeesOwed() (gas: 416416)
[PASS] test_LossWithdrawalDoesNotChargeFees() (gas: 380529)
[PASS] test_MaxFeeRateEnforced() (gas: 16152)
[PASS] test_MixedRewardAndDepositShares_WithdrawalRetainsRemainingRewardShares() (gas: 407520)
[PASS] test_PartialWithdrawWithVaultTokenFees_DoesNotDoubleChargeProfit() (gas: 448684)
[PASS] test_ProfitWithdrawalAccruesFees() (gas: 405048)
[PASS] test_RecordYieldAccruesFees() (gas: 391950)
[PASS] test_YieldTokenFeeTrackedAndConvertedOnSwap() (gas: 399348)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 2.21ms (473.04µs CPU time)

Ran 42 tests for test/unit/AdapterRegistry.t.sol:AdapterRegistryTest
[PASS] test_AdminRole_CanRegisterAdapters() (gas: 81563)
[PASS] test_AdminRole_Verification() (gas: 24003)
[PASS] test_BatchOperations_AtomicFailure() (gas: 236019)
[PASS] test_EmergencyRole_CanPauseAndUnregister() (gas: 95498)
[PASS] test_GetAllTargets_AfterRemoval() (gas: 210330)
[PASS] test_GetAllTargets_Empty() (gas: 8969)
[PASS] test_GetAllTargets_FilterUnregistered() (gas: 315282)
[PASS] test_GetAllTargets_MultipleTargets() (gas: 329967)
[PASS] test_GetTargetAdapter_NoMapping() (gas: 16507)
[PASS] test_GetTargetAdapter_ValidMapping() (gas: 178683)
[PASS] test_IsRegistered_Correct() (gas: 83228)
[PASS] test_Pause_OnlyEmergency() (gas: 37409)
[PASS] test_Pause_OnlyEmergencyRole() (gas: 13600)
[PASS] test_QueryDuringPause_Blocked() (gas: 206627)
[PASS] test_RegisterAdapter_AlreadyRegistered() (gas: 86563)
[PASS] test_RegisterAdapter_EOA() (gas: 18443)
[PASS] test_RegisterAdapter_EmitsEvent() (gas: 81928)
[PASS] test_RegisterAdapter_OnlyAdmin() (gas: 58020)
[PASS] test_RegisterAdapter_Success() (gas: 84211)
[PASS] test_RegisterAdapter_ZeroAddress() (gas: 14141)
[PASS] test_RemoveTarget_NotRegistered() (gas: 21117)
[PASS] test_RemoveTarget_OnlyEmergency() (gas: 181766)
[PASS] test_RemoveTarget_Success() (gas: 147792)
[PASS] test_RenounceRole_Success() (gas: 66249)
[PASS] test_RoleManagement_AdminCanPerformAdminActions() (gas: 193155)
[PASS] test_RoleManagement_EmergencyCanPerformEmergencyActions() (gas: 190098)
[PASS] test_RoleManagement_NonAdminCannotPerformAdminActions() (gas: 67766)
[PASS] test_RoleManagement_NonEmergencyCannotPerformEmergencyActions() (gas: 189537)
[PASS] test_RoleManagement_RoleAdminRelationships() (gas: 12385)
[PASS] test_RoleManagement_RoleQueries() (gas: 33186)
[PASS] test_SetTargetAdapter_OnlyAdmin() (gas: 88929)
[PASS] test_SetTargetAdapter_SameAdapter() (gas: 181636)
[PASS] test_SetTargetAdapter_Success() (gas: 181183)
[PASS] test_SetTargetAdapter_UnregisteredAdapter() (gas: 61827)
```

```
[PASS] test_SetTargetAdapter_UpdateExisting() (gas: 258694)
[PASS] test_SetTargetAdapter_ZeroTarget() (gas: 83381)
[PASS] test_Unpause_OnlyAdmin() (gas: 32006)
[PASS] test_Unpause_OnlyAdminRole() (gas: 43076)
[PASS] test_UnregisterAdapter_AffectsGetTargetAdapter() (gas: 167829)
[PASS] test_UnregisterAdapter_NotRegistered() (gas: 60221)
[PASS] test_UnregisterAdapter_OnlyEmergency() (gas: 86892)
[PASS] test_UnregisterAdapter_Success() (gas: 72420)
Suite result: ok. 42 passed; 0 failed; 0 skipped; finished in 5.70ms (3.36ms CPU time)

Ran 6 tests for test/integration/MultiProtocolAdapterIntegration.t.sol:MultiProtocolAdapterIntegrationTest
[PASS] test_AaveV3_DepositWithdraw() (gas: 586256)
[PASS] test_CompoundV2_DepositWithdraw() (gas: 582717)
[PASS] test_CompoundV3_DepositWithdraw() (gas: 576504)
[PASS] test_ConsolidateFromAllProtocols() (gas: 831268)
[PASS] test_DistributeAcrossAllProtocols() (gas: 849286)
[PASS] test_MovePositionAcrossProtocols() (gas: 635360)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.41ms (1.44ms CPU time)

Ran 6 tests for test/security/StressSecurity.t.sol:StressSecurityTest
[PASS] test_BatchWithPausedRegistryReverts() (gas: 456172)
[PASS] test_GasHeavySequentialDeposits() (gas: 807667)
[PASS] test_LargeBatchExecution_Succeeds() (gas: 676394)
[PASS] test_LargeValueDepositAndWithdraw() (gas: 495843)
[PASS] test_MassWalletCreation_NonOperatorReverts() (gas: 15779)
[PASS] test_MassWalletCreation_OperatorOnly() (gas: 6544977)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 4.12ms (3.48ms CPU time)

Ran 35 tests for test/unit/AgentWalletFactory.t.sol:AgentWalletFactoryTest
[PASS] test_AccessControl_EventEmission() (gas: 55806)
[PASS] test_AdminOnly_AllFunctions() (gas: 96511)
[PASS] test_ComputeWalletAddress_Deterministic() (gas: 16430)
[PASS] test_ComputeWalletAddress_DifferentSalts() (gas: 20398)
[PASS] test_ComputeWalletAddress_SameInputs() (gas: 16221)
[PASS] test_CreateAgentWallet_DuplicateRevert() (gas: 197393)
[PASS] test_CreateAgentWallet_EmitsEvent() (gas: 199446)
[PASS] test_CreateAgentWallet_Success() (gas: 202605)
[PASS] test_CreateWallet_MaxGasUsage() (gas: 191907)
[PASS] test_CreateWallet_NonContractAsset() (gas: 75905)
[PASS] test_CreateWallet_OnlyOperators() (gas: 149953)
[PASS] test_CreateWallet_ZeroAsset() (gas: 71485)
[PASS] test_CreateWallet_ZeroOwner() (gas: 71266)
[PASS] test_GetConfiguration_Correct() (gas: 21079)
[PASS] test_GetWalletsByOwner_Correct() (gas: 253429)
[PASS] test_GrantOperatorRole_OnlyAdmin() (gas: 18762)
[PASS] test_GrantOperatorRole_RegrantAtLimit_Succeeds() (gas: 351740)
[PASS] test_GrantOperatorRole_Success() (gas: 68749)
[PASS] test_HasOperatorRole_Correct() (gas: 68451)
[PASS] test_ImplementationImmutable_AfterDeploy() (gas: 200132)
[PASS] test_OperatorOnly_CreateFunction() (gas: 256350)
[PASS] test_OperatorRole_EmitsEvents() (gas: 56264)
[PASS] test_OwnerWalletCount_Correct() (gas: 353019)
[PASS] test_PausedState_BlockOperations() (gas: 80269)
[PASS] test_RevokeOperatorRole_Success() (gas: 56377)
[PASS] test_SetAdapterRegistry_Success() (gas: 27148)
[PASS] test_SetConfiguration_ZeroAddress() (gas: 23983)
[PASS] test_SetFeeTracker_Success() (gas: 27830)
[PASS] test_SetImplementation_OnlyAdmin() (gas: 16094)
[PASS] test_SetImplementation_Success() (gas: 27306)
[PASS] test_StateConsistency_MultipleCreations() (gas: 353396)
[PASS] test_UnpausedState_ResumeOperations() (gas: 201131)
[PASS] test_WalletCounter_Increments() (gas: 348336)
[PASS] test_WalletCounter_Persistent() (gas: 194300)
[PASS] test_WalletExists_Correct() (gas: 196589)
Suite result: ok. 35 passed; 0 failed; 0 skipped; finished in 5.62ms (3.13ms CPU time)

Ran 9 tests for test/integration/YieldSeekerE2E.t.sol:YieldSeekerE2ETest
[PASS] test_AccessControl_BlockingIsOwnerPrerogative() (gas: 373891)
[PASS] test_AccessControl_OwnerCanControlWallet() (gas: 537765)
[PASS] test_ComplexRebalancingStrategy() (gas: 559544)
[PASS] test_HighFrequencyTrading_RapidDepositWithdrawals() (gas: 1692336)
[PASS] test_LargeScaleOperations() (gas: 474784)
[PASS] test_MultiAgentScenario_SharedVault() (gas: 1334060)
[PASS] test_RecoveryFromBlockedAdapter() (gas: 568800)
[PASS] test_RecoveryFromRegistryPause() (gas: 576846)
```

```
[PASS] test_UserWorkflow_OnboardingToActiveTrading() (gas: 942101)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 8.38ms (3.82ms CPU time)

Ran 54 tests for test/integration/AgentWallet.t.sol:AgentWalletIntegrationTest
[PASS] test_AdapterBlocking_CrossContract() (gas: 433605)
[PASS] test_AdapterBlocking_PreventsUserFromUsing() (gas: 583951)
[PASS] test_AdapterExecution_WithInsufficientBalance() (gas: 454961)
[PASS] test_AdapterIntegration_MultipleUsers() (gas: 1001942)
[PASS] test_AdapterUnblock_RestoresAccess() (gas: 556875)
[PASS] test_AdapterUnregistration_ImmediateEffect() (gas: 563771)
[PASS] test_BatchExecution_AtomicFailure() (gas: 585069)
[PASS] test_BatchExecution_LargeScale() (gas: 788344)
[PASS] test_CompleteYieldCycle() (gas: 513230)
[PASS] test_ConsecutiveDeposits_Accumulate() (gas: 578129)
[PASS] test_CreateFreshV2Wallet_InitializesCorrectly() (gas: 3119931)
[PASS] test_CreateWalletViaFactory_Success() (gas: 382986)
[PASS] test_CreateWallet_DeterministicAddress() (gas: 361990)
[PASS] test_CreateWallet_DuplicateIndex() (gas: 362810)
[PASS] test_CreateWallet_InvalidAsset() (gas: 18938)
[PASS] test_CreateWallet_RealImplementationUsed() (gas: 549634)
[PASS] test_CreateWallet_UnauthorizedOperator() (gas: 17906)
[PASS] test_CreateWallet_WithDifferentAgentIndices() (gas: 683961)
[PASS] test_CreateWallet_ZeroOwner() (gas: 18412)
[PASS] test_DepositAndWithdraw_RealSequence() (gas: 478559)
[PASS] test_DepositPercentage_CalculatesCorrectly() (gas: 553091)
[PASS] test_DepositThroughAdapterRecordsFeeTracking() (gas: 530308)
[PASS] test_DirectVaultShareTransfer_CausesUnderflow() (gas: 583716)
[PASS] test_ExecuteViaAdapterBatch_MultipleOperations() (gas: 580994)
[PASS] test_ExecuteViaAdapter_OnlyExecutorsAllowed() (gas: 403309)
[PASS] test_ExecuteViaAdapter_OperatorCanExecute() (gas: 550204)
[PASS] test_ExecuteViaAdapter_OwnerCanExecute() (gas: 550404)
[PASS] test_ExecuteViaAdapter_RealRegistry() (gas: 550836)
[PASS] test_FactoryComponents_ProperlySet() (gas: 12299)
[PASS] test_FactoryRegistryIntegration() (gas: 555759)
[PASS] test_Factory_UpdateImplementation() (gas: 2121845)
[PASS] test_InitialState_CorrectConfiguration() (gas: 359543)
[PASS] test_InitializationEvent_RealEmission() (gas: 365170)
[PASS] test_InitializeWalletWithRealContracts() (gas: 367466)
[PASS] test_InitializeWithRegistry_Correct() (gas: 367732)
[PASS] test_MultiVaultInteraction() (gas: 1783091)
[PASS] test_MultipleOperators_CanCreateWallets() (gas: 484511)
[PASS] test_MultipleWallets_SharedRegistry() (gas: 1005733)
[PASS] test_PartialWithdraw_LeavesRemainingShares() (gas: 579965)
[PASS] test_RebalancingFlow() (gas: 578128)
[PASS] test_RegistryPause_EffectsOnWallet() (gas: 575002)
[PASS] test_TargetBlocking_PreventsUserFromUsingTarget() (gas: 583905)
[PASS] test_UpgradeWallet_WithActivePosition_FeesTrackedCorrectly() (gas: 3130408)
[PASS] test_VaultShareReward_FeesTrackedCorrectly() (gas: 1887756)
[PASS] test_WalletCreatedEvent_EmitsCorrectly() (gas: 365264)
[PASS] test_WalletFunctionality_AfterSync() (gas: 543091)
[PASS] test_WalletProxy_ImplementationUpgrade() (gas: 549810)
[PASS] test_WithdrawAllAssetToUser_BaseAsset_RespectsFees() (gas: 560703)
[PASS] test_WithdrawAssetToUser_BaseAsset_RespectsFees() (gas: 565901)
[PASS] test_WithdrawAssetToUser_NonBaseAsset_Reverts() (gas: 858455)
[PASS] test_WithdrawAssetToUser_RevertsOnZeroAddress() (gas: 404405)
[PASS] test_WithdrawThroughAdapterRecordsFeeTracking() (gas: 478439)
[PASS] test_WithdrawToUser_ThroughWallet() (gas: 411308)
[PASS] test_YieldGeneration_WithRealFeeTracking() (gas: 513872)
Suite result: ok. 54 passed; 0 failed; 0 skipped; finished in 9.38ms (22.26ms CPU time)

Ran 3 tests for test/unit/FeeTracker.t.sol:YieldSeekerFeeTrackerTest
[PASS] test_RecordAgentVaultShareWithdraw_DoubleChargingOnUntrackedSharesWithFeeOwed() (gas: 96047)
[PASS] test_VaultWithdrawal_SimpleYieldCalculation() (gas: 75808)
[PASS] test_VaultWithdrawal_WithYieldTokenFees() (gas: 92472)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.73ms (538.67µs CPU time)

Ran 65 tests for test/unit/AgentWalletV1.t.sol:AgentWalletV1Test
[PASS] test_BlockAdapter_AlreadyBlocked() (gas: 42948)
[PASS] test_BlockAdapter_EmitsEvent() (gas: 40319)
[PASS] test_BlockAdapter_OnlyOwner() (gas: 16216)
[PASS] test_BlockAdapter_Success() (gas: 41090)
[PASS] test_BlockTarget_EmitsEvent() (gas: 39688)
[PASS] test_BlockTarget_Success() (gas: 42194)
[PASS] test_ExecuteViaAdapterBatch_EmptyBatch() (gas: 18931)
[PASS] test_ExecuteViaAdapterBatch_GasLimits() (gas: 202266)
```

```
[PASS] test_ExecuteViaAdapterBatch_MismatchedArrays() (gas: 18530)
[PASS] test_ExecuteViaAdapterBatch_OnlyExecutors() (gas: 18561)
[PASS] test_ExecuteViaAdapterBatch_PartialFailure() (gas: 93665)
[PASS] test_ExecuteViaAdapterBatch_ValidBatch() (gas: 96187)
[PASS] test_ExecuteViaAdapter_BlockedAdapter() (gas: 76656)
[PASS] test_ExecuteViaAdapter_BlockedTarget() (gas: 78416)
[PASS] test_ExecuteViaAdapter_InvalidTarget() (gas: 51922)
[PASS] test_ExecuteViaAdapter_OnlyExecutors() (gas: 19739)
[PASS] test_ExecuteViaAdapter_ReturnData() (gas: 54988)
[PASS] test_ExecuteViaAdapter_RevertData() (gas: 55751)
[PASS] test_ExecuteViaAdapter_UnregisteredAdapter() (gas: 30237)
[PASS] test_ExecuteViaAdapter_ValidAdapter() (gas: 54891)
[PASS] test_IsAdapterBlocked_Correct() (gas: 32245)
[PASS] test_IsTargetBlocked_Correct() (gas: 33128)
[PASS] test_SyncFromFactory_EmitsEvent() (gas: 30160)
[PASS] test_SyncFromFactory_OnlySyncers() (gas: 17320)
[PASS] test_SyncFromFactory_Success() (gas: 32076)
[PASS] test_SyncFromFactory_UpdatesFeeTracker() (gas: 28133)
[PASS] test_SyncFromFactory_UpdatesRegistry() (gas: 28337)
[PASS] test_UnblockAdapter_EmitsEvent() (gas: 30957)
[PASS] test_UnblockAdapter_NotBlocked() (gas: 20013)
[PASS] test_UnblockAdapter_Success() (gas: 32708)
[PASS] test_UnblockTarget_Success() (gas: 33064)
[PASS] test_Upgrade_EmitsEvent() (gas: 15609)
[PASS] test_Upgrade_InvalidImplementation() (gas: 15213)
[PASS] test_Upgrade_OnlyOwner() (gas: 15710)
[PASS] test_Upgrade_PreserveERC7201Storage() (gas: 45920)
[PASS] test_Upgrade_PreserveStorage() (gas: 24068)
[PASS] test_Upgrade_ValidImplementation() (gas: 16252)
[PASS] test_ValidateUserOp_EmptySignature() (gas: 13444)
[PASS] test_ValidateUserOp_InsufficientGas() (gas: 14893)
[PASS] test_ValidateUserOp_MalformedSignature() (gas: 13736)
[PASS] test_ValidateUserOp_ReplayAttack() (gas: 15685)
[PASS] test_ValidateUserOp_WrongOwner() (gas: 15018)
[PASS] test_WithdrawAllAssetToUser_BaseAsset_RespectsFees() (gas: 132112)
[PASS] test_WithdrawAllAssetToUser_EmptyBalance() (gas: 39727)
[PASS] test_WithdrawAllAssetToUser_NonBaseAsset_WithdrawsAll() (gas: 556638)
[PASS] test_WithdrawAllAssetToUser_OnlyOwner() (gas: 69185)
[PASS] test_WithdrawAllAssetToUser_ZeroAsset() (gas: 17546)
[PASS] test_WithdrawAllAssetToUser_ZeroRecipient() (gas: 65796)
[PASS] test_WithdrawAssetToUser_BaseAsset_RespectsFees() (gas: 142671)
[PASS] test_WithdrawAssetToUser_BaseAsset_ValidAmount() (gas: 90382)
[PASS] test_WithdrawAssetToUser_InsufficientBalance() (gas: 34483)
[PASS] test_WithdrawAssetToUser_NonBaseAsset_NoFees() (gas: 555662)
[PASS] test_WithdrawAssetToUser_OnlyOwner() (gas: 68650)
[PASS] test_WithdrawAssetToUser_ZeroAsset() (gas: 17312)
[PASS] test_WithdrawAssetToUser_ZeroRecipient() (gas: 66996)
[PASS] test_WithdrawEthToUser_EmitsEvent() (gas: 53229)
[PASS] test_WithdrawEthToUser_InsufficientBalance() (gas: 17608)
[PASS] test_WithdrawEthToUser_OnlyOwner() (gas: 15959)
[PASS] test_WithdrawEthToUser_ValidAmount() (gas: 53559)
[PASS] test_WithdrawEthToUser_ZeroAmount() (gas: 17826)
[PASS] test_WithdrawTokenToUser_EmitsEvent() (gas: 82130)
[PASS] test_WithdrawTokenToUser_InsufficientBalance() (gas: 24312)
[PASS] test_WithdrawTokenToUser_OnlyOwner() (gas: 16594)
[PASS] test_WithdrawTokenToUser_ValidAmount() (gas: 83171)
[PASS] test_WithdrawTokenToUser_ZeroAmount() (gas: 25651)
Suite result: ok. 65 passed; 0 failed; 0 skipped; finished in 5.68ms (12.90ms CPU time)

Ran 16 tests for test/integration/SystemIntegration.t.sol:SystemIntegrationTest
[PASS] test_AtomicBatchExecution_PartialFailure() (gas: 615487)
[PASS] test_BatchExecution_RealAdapters() (gas: 581850)
[PASS] test_CompleteRebalance_MultipleVaults() (gas: 2203471)
[PASS] test_CreateWallet_BasicFlow() (gas: 360925)
[PASS] test_EmergencyUnregistration_ImmediateEffect() (gas: 565028)
[PASS] test_FeeCalculation_MultipleYieldEvents() (gas: 398386)
[PASS] test_FullSystemSetup_Success() (gas: 31263)
[PASS] test_MultipleWalletsPerUser_IndependentState() (gas: 913436)
[PASS] test_MultipleWallets_SharedInfrastructure() (gas: 1006082)
[PASS] test_RegistryPause_BlocksExecution() (gas: 573351)
[PASS] test_RegistryUpdate_AffectsActiveWallets() (gas: 1669028)
[PASS] test_UserBlocklist_IndependentFromRegistry() (gas: 567587)
[PASS] test_WalletAdapterExecution_BasicFlow() (gas: 551687)
[PASS] test_WalletSync_UpdatesConfiguration() (gas: 1939297)
[PASS] test_WalletUpgrade_PreservesState() (gas: 2674134)
```

```
[PASS] test_YieldGeneration_WithFeeTracking() (gas: 606088)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 9.46ms (3.31ms CPU time)

Ran 23 test suites in 118.84ms (78.91ms CPU time): 373 tests passed, 0 failed, 0 skipped (373 total tests)
```

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at https://app.auditagent.nethermind.io.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.